LEAD UNIVERSITY

Administración Moderna de Memoria

Justificación

- El multiprocesamiento require un manejo de memoria más complejo
- No es possible preveer la demanda de memoria de un proceso: normalmente siempre requiren más y más y más…
- Se necesita tener mútiples procesos corriendo

Todo lo anterior, entre otras razones, pero sobre todo SUMADO A LA LIMITACIÓN DE MEMORIA RAM que siempre ha sido un problema (por costo o por disponibilidad) requería ser más “creativos” con la memoria.

¿Cómo afrontar estos retos?

- Hay 4 grandes tendencias
  - Paginación
  - Paginas por demanda
  - Segmentación de Memoria
  - Paginas sementadas por demanda, éste ultimo es el fundamento de la Memoria Virtual Moderna.

Paginación de Memoria (1)

- Aquí aparece la idea de PAGINA
  - Cuando un programa/trabajo es cargado en memoria, es dividido en partes llamadas PÁGINAS, las cuales se cargan en PAGE FRAMES (“MARCOS DE PÁGINA”)

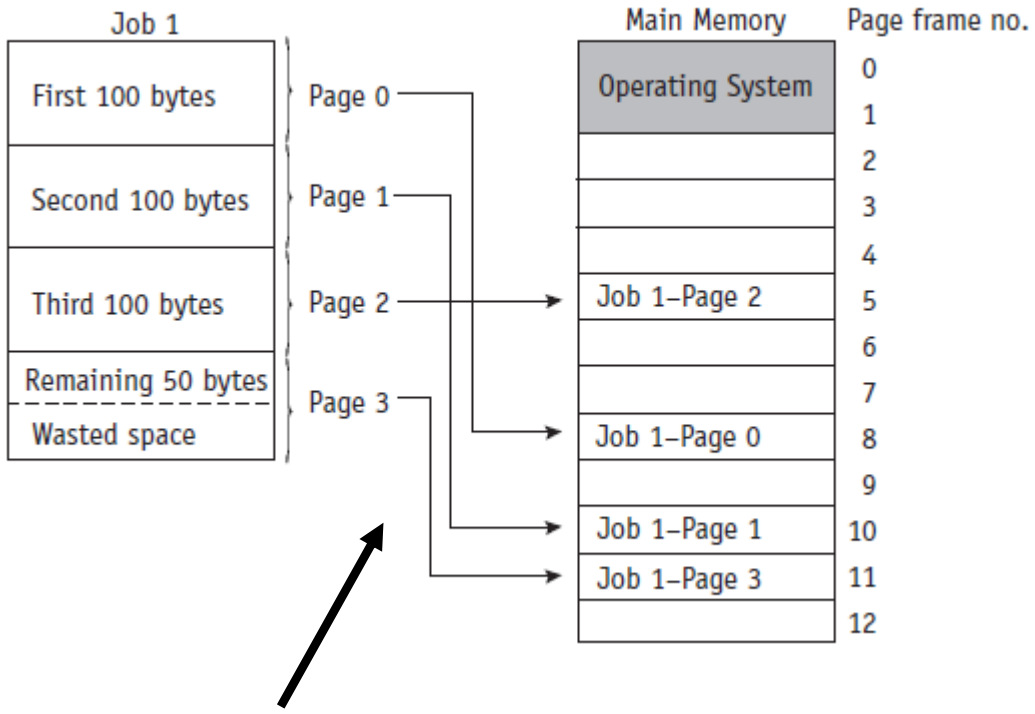
- La tendencia es que cada página sea del tamaño de un bloque de memoria (un conjunto de direcciones de memoria) y de un SECTOR del disco (un sector es una parte en que es dividida la memoria física o discos). Cuando los 3 son del mismo tamaño es más fácil mover información entre ellos.

Paginación de Memoria(2)

- Cuando un programa se va a ejecutar, el Memory Manager:
  - Determina cuantas páginas necesitará
  - Localiza los page frames en la memoria principal
  - Carga el programa ahi.

- Al usar páginas, NO TODO EL PROGRAMA ESTÁ CARGADO SECUENCIALMENTE EN MEMORIA.
  - Esto tiene la ventaja de se hace un uso *más eficiente de memoria* al evitar los problemas de fragmentación
  - Y la desventaja de que *ahora el trabajo del memory manager es más complejo porque necesita saber donde está cada parte del programa*

Paginación de Memoria (3)



En lugar de estas líneas, el Memory Manager usa TABLAS

Vemos el caso hipotético de un proceso que requiere 350k, y las páginas son de 100k.

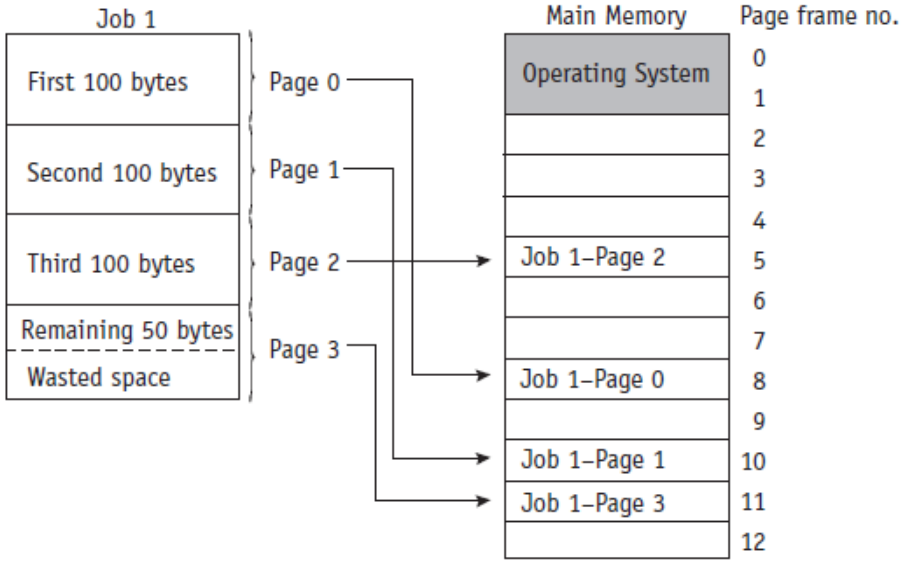
Ahora al SO le quedan libres solo 700k disponibles aunque tenga 750 reales libres.

Aquí empezamos a perfilar la diferencia entre las cosas lógicamente disponibles y las físicamente disponibles (que no necesariamente es lo mismo)

Tablas del Memory Manager

- Se usan 3 tablas (los nombres pueden variar entre SO).
  - Job Table: para cada proceso activo, contiene el tamaño del proceso y la posición en la Page Map Table donde está guardado.
  - Page Map Table: esta tiene la lista de páginas de cada proceso CONSECUTIVAMENTE, con su número (0,1,2,..) y su “dirección” de inicio.
  - Memory Map Table: esta tabla tiene TODAS las páginas y su estatus: libre/ocupado.

Tablas del Memory Manager



Job Table

Job1 150

Memory Map Table

1 0
1 100
0 200
0 300
0 400
1 500
0 600
0 700
1 800

Page Map Table

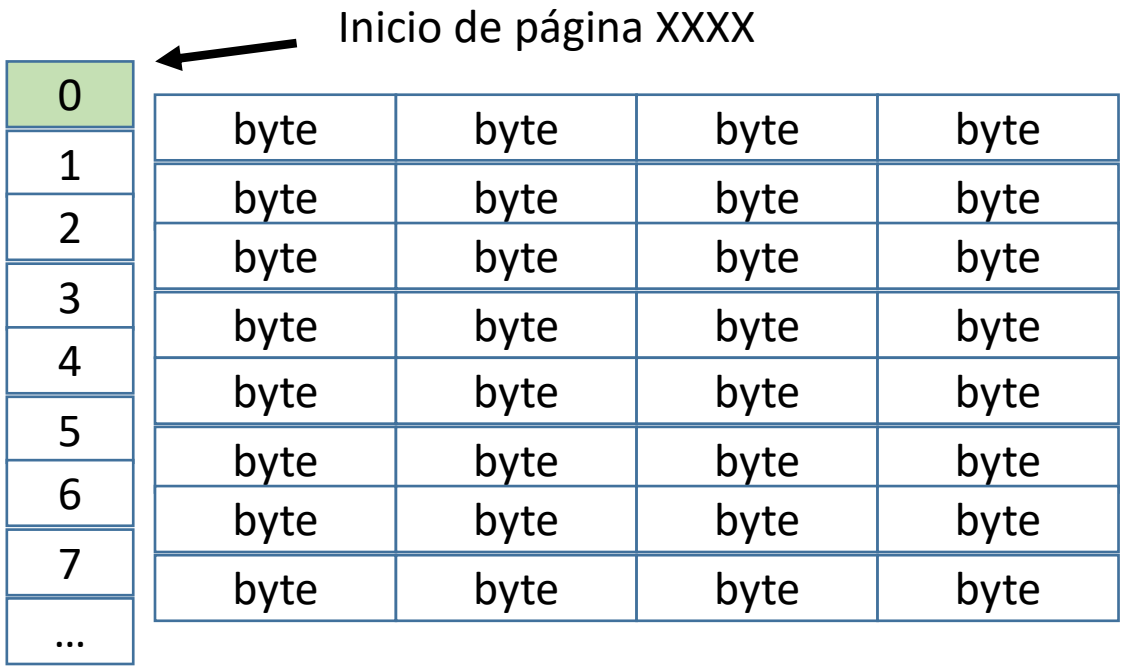
….	
150	1 800
151	2 1000
152	3 500
153	4 1100
….	

Desplazamiento/Offset

Recordemos que en la memoria se guardan DATOS o INSTRUCCIONES.

Significa que el SO debe saber CÓMO ACCEDER a ellos dentro de una página

El OFFSET o DESPLAZAMIENTO es cuantos bytes o palabras tengo que moverme a partir del inicio de la página para llegar al dato/instrucción que se necesita.



Y si, eso es tarea del Memory Manager.
Por eso al construir compiladores y SO hay que considerar: si la memoria es byte direccionable, tamaño de bus, tamaño de palabra, tamaño de registros…

COMO LAS PÁGINAS SON “LÓGICAS” (RELATIVAS) el SO debe convertirlas a direcciones reales FISICAS cada vez que necesita una instrucción o dato. Este proceso se llama RESOLUCIÓN DE DIRECCIÓN (Address Resolution)

Paginación de Memoria (y 6)

- La paginación de memoria limita el problema de fragmentación a la última página del proceso.

- El éxito de este modelo depende DEL TAMAÑO DE LA PÁGINA. Un tamaño que sirve un dia para ciertos procesos puede no server para otros. Se depende demasiado del tamaño de los procesos.

Paginación/Páginas por Demanda (1)

- La idea es CARGAR EN MEMORIA SOLO LA PARTE DEL PROGRAMA REQUERIDA PARA EL PROCESO.
  - Fue el primer modelo que removió la idea de cargar todo en memoria.

- Los programas si se dividen en páginas pero solo se trae a la memoria principal el fragmento de código o datos necearios.
  - Se trata de aprovechar la característica de la arquitectura de Von Neumann = ejecución secuencial → escritura secuencial.
  - Aparece la idea de memoria virtual.
  - Los procesos necesitan MENOS MEMORIA para ejecutarse.

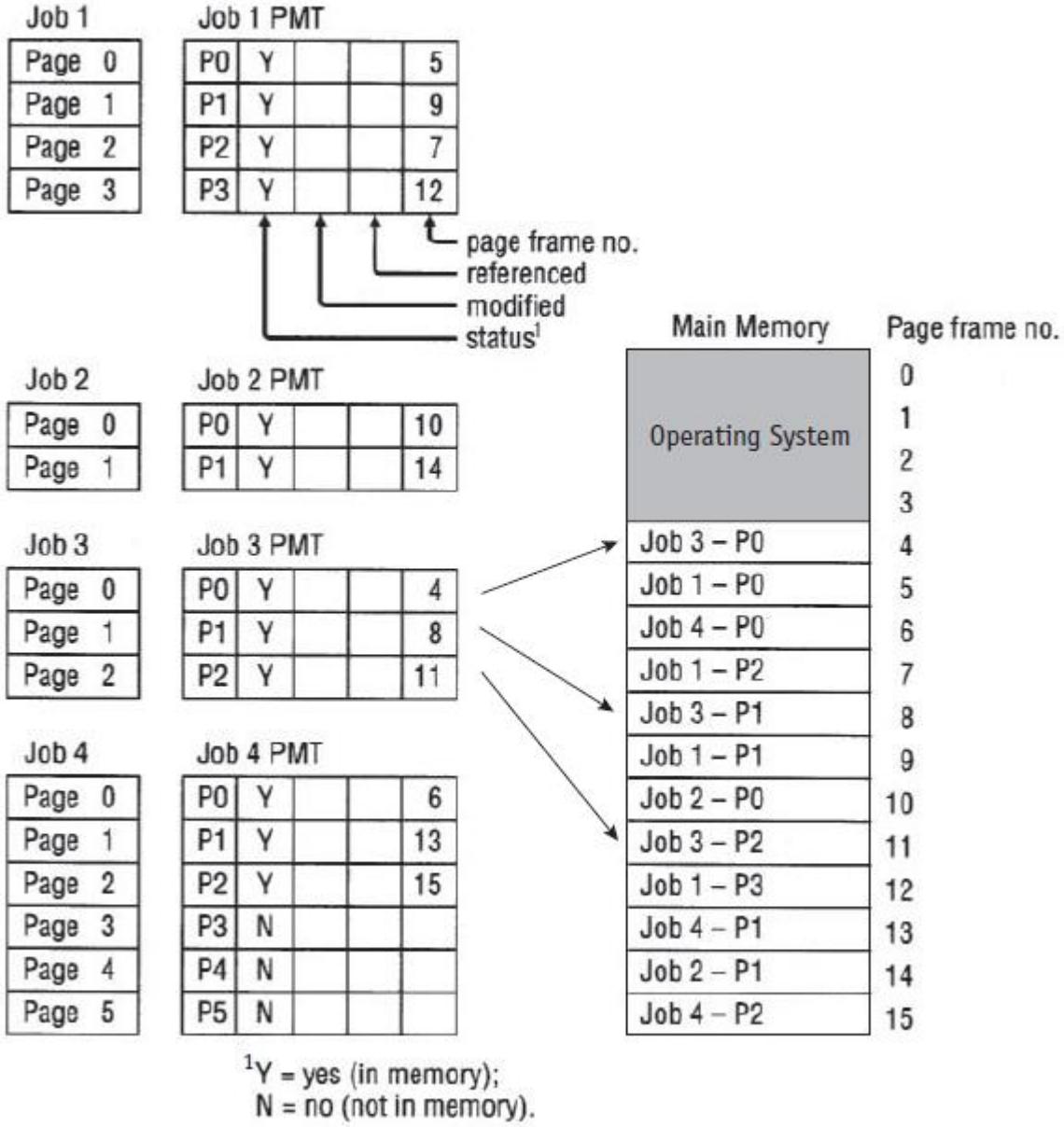
Paginación/Páginas por Demanda (2)

- Este modelo es posibilitado por la existencia de buses que permiten la comunicación entre la CPU y los dispositivos de almacenamiento fisico
  - IDE/SCSI/PCI-express
  - Memorias Flash/USB
- Para que este estilo funcione los datos deben moverse RÁPIDAMENTE entre Memoria Ram y Memoria Fisica.
  - Este proceso se conoce como ***SWAPPING:*** las políticas de swapping determinan cómo se hacen los intercambios

Tablas del SO para Páginas/Paginación por Demanda

- La única diferencia en las tablas de administración de memoria es en el PMT, que tien que tener ahora 3 nuevos campos:
  - Una bandera que indique que si la página está en memoria o no: si se necesita la página, se inicia la operación de swap.
  - Una bandera que indique si los contenidos de la página han sido modificados o no: si esta bandera indica que hubo cambios, implica que hay que re-escribir la página en el almacenamiento fisico.
  - Una bandera que indique si la página ha sido accedida recientemente. Básicamente si una página no se esta accediendo, es mejor descargarla de memoria.

Tablas del Memory Manager



Políticas de Sustitución de Páginas para Paginación por Demanda

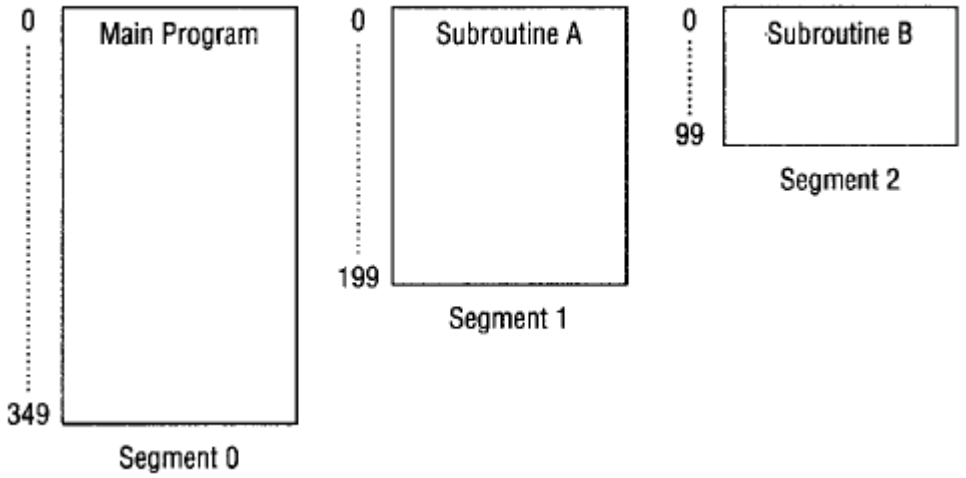
- Hay 2 grandes formas de administrar páginas
  - First In/First Out: “la mejor candidata para salir de la memoria es la página que tiene más tiempo en memoria”. O sea, como si cargar las páginas en memoria se hiciera como una cola. Aunque es un esquema muy simple, tiende a dar muchos fallos en programas largos.
  - Least Recently Used: “la mejor candidate para salir de la memoria es la página que menos han accedido en los últimos tiempos”. O sea, se da preferencia al uso. Existen varias variantes de este esquema; su principal problema es ser más dificil de implementar; pero es el más usado.

Segmentación de Memoria (1)

- La segmentación tiene su origen en aprovechar el uso de bibliotecas, módulos, componentes, etc. Es decir de la modularización del Software.
- Cada proceso es dividido en segmentos de DIFERENTES TAMAÑOS: esto se hace por sub-rutinas, objetos, bibliotecas.
- En este método, NO SE DIVIDE LA MEMORIA EN PAGE FRAMES. De hecho, DESAPARECE EL CONCEPTO DE PÁGINAS
- Requiere trabajo de los COMPILADORES.

Segmentación de Memoria (2)

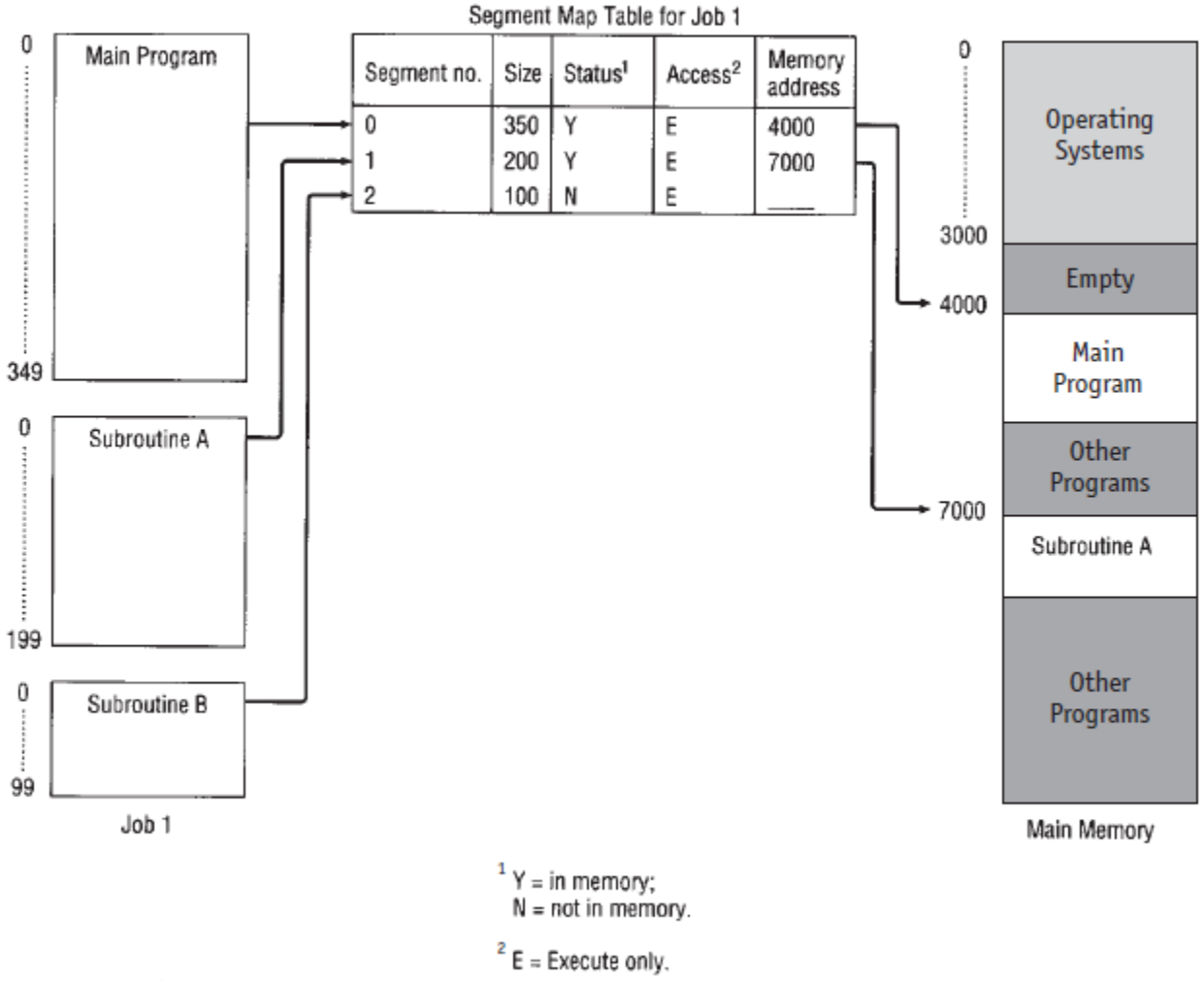
- Cuando se compila un programa, se divide sus secciones de código que actuarán como SEGMENTOS.



De esta forma, el Memory Manager necesita controlar LOS SEGMENTOS y su carga.

Las páginas que se cargan entonces son estrictamente del tamaño de los segmentos.

Segmentación de Memoria (3)



El Memory Manager ahora para CADA PROCESO, debe mantener una nueva tabla llamada SEGMENT MAP TABLE (Tabla de Mapa de Segmentos). Desaparece el concepto de Page Map Table.

Para cada segmento se indica el número, tamaño, status, y ubicación en memoria.

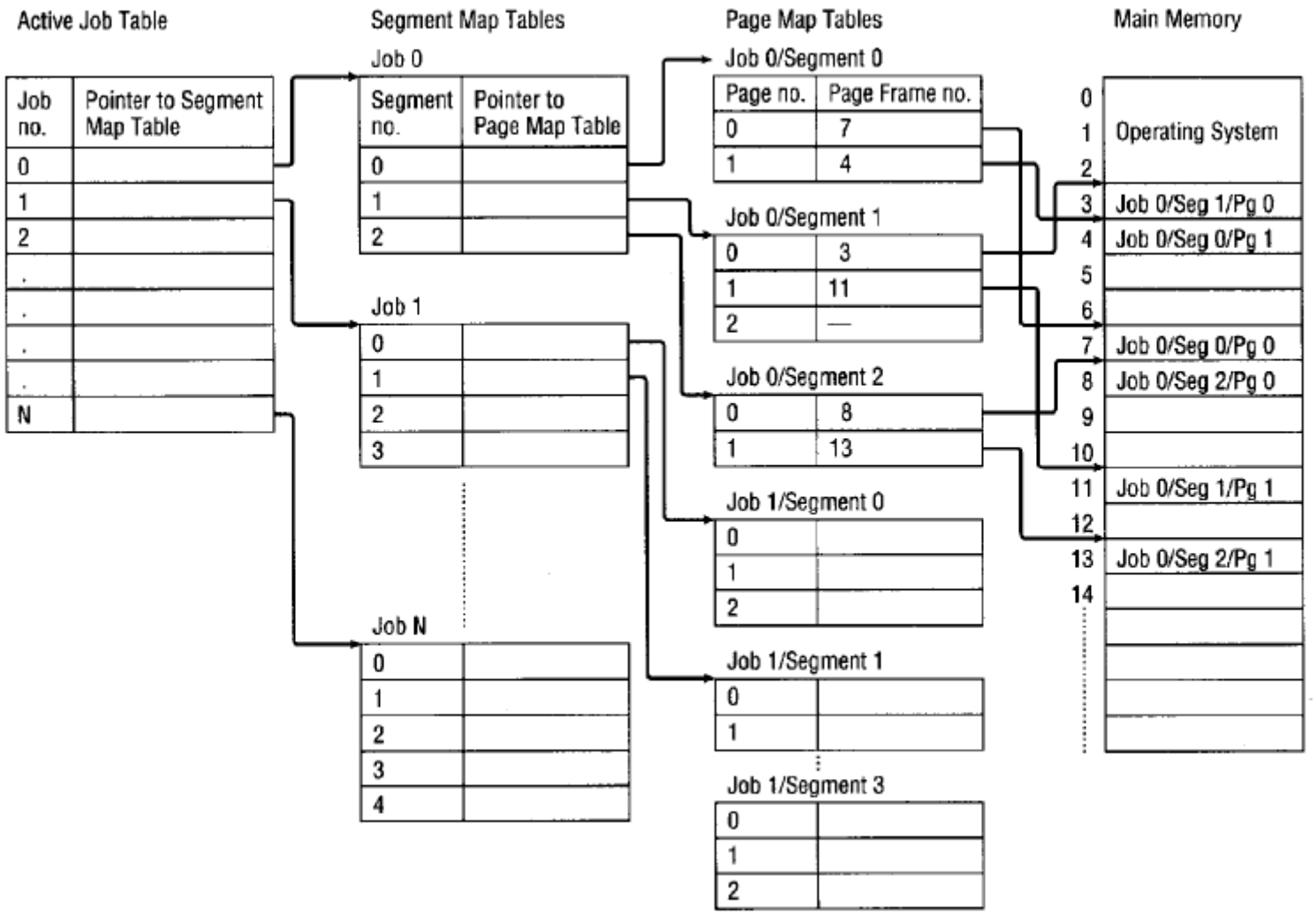
El problema de la segmentación es que se incrementa mucho la fragmentación externa (es decir, memoria libre muy fragmentada que no se puede usar)

Páginas Segmentadas por Demanda (1)

- Es una evolución de los últimos 2 esquemas: crear páginas por demanda y colocar en ella segmentos.

- La idea central es dividir los segmentos en páginas del mismo tamaño. El principal problema es que la administración es más compleja tanto en Hardware como Software

Páginas Segmentadas por Demanda (2)



Otro problema de este esquema es el tiempo necesario para localizar segmentos interdependientes.

Para minimizar esto, se creo la MEMORIA ASOCIATIVA.

La Memoria Asociativa son registros extra de cada proceso, que se cargan y permiten que si un segmento X necesita trabajar con otros segmentos, pues se cargan en memoria dichos segmentos.

Memoria Virtual (1)

- Recordemos que esto se refiere a la capacidad de guardar parte de lo que deberia ser memoria ocupada por el proceso en Memoria Principal en memoria Fisica.

- Originalmente los programas y su tamaño eran vigilados por los programadores: incluso un sistema se podia dividir en varios programas.

- Se require colaboración entre el Memory Manager, el hardware alrededor del procesador (buses, controladores, interrupciones). Es algo que cubre todo el diseño del Sistema Computacional.

Memoria Virtual (y 2)

- El diseño e implementación es complejo y COSTOSO, pero entre las ventajas de usar este esquema están:
  - Es muy difícil alcanzar un límite real para el tamaño de los programas.
  - Se usa más eficientemente la memoria principal.
  - La multiprogramación es potenciada.